QALight

Frontend-розробка

Заняття 22. Promise, HTTP-запити.

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Promise, HTTP-запити.
Проміси. Споживачі: then, catch. Метод fetch.
Використання методу fetch.

Promise, HTTP-запити.

Проміси. Споживачі: then, catch. Метод fetch.

Об’єкт Promise

Теорія: promise

Є код–“**виробник**”, котрий робить щось, що займає час. Наприклад, завантажує дані з мережі.
Є код–“**споживач**”, котрий очікує отримати результат від коду-“виробника”, як тільки-но результат буде готовий. Цей результат може знадобитися багатьом функціям.

Проміс (англ. Promise) – це спеціальний об’єкт в JavaScript, котрий зв’язує “виробника” і “споживача” разом. Код-“виробник” виконується стільки часу, скільки потрібно щоб отримати результат, а проміс робить результат доступним для коду-“споживача”, який на нього підсився, як тільки-но код-“виробник” поверне результат.

Проміс - це спеціальний об’єкт, який представляє кінцевий результат асинхронної операції. Він діє як **проксі результату операції**. До появи обіцянок JavaScript кращим способом роботи з асинхронною операцією було використання **зворотного виклику setTimeout**.

Обіцянка (**Promise**) означає трансформацію. Це **перетворення** функції, яка приймає **callback**, **у функцію, яка повертає обіцянку**.

Створення проміса

Теорія: promise

Синтаксис створення проміса:

```
let promise = new Promise(function(resolve, reject) {
 // код, котрий поверне результат (по аналогії код–"виробник", код-"співак")
});
```

Функція передана в new Promise називається **виконавцем**. Коли створюється new Promise вона запускається автоматично. В ній знаходиться код “**виробник**”, котрий зрештою поверне результат.

Аргументи функції виконавця resolve і reject – це колбеки, які надає сам JavaScript. Наш код має бути тільки всередині функції виконавця.

Об’єкт **Promise** після завершення своєї роботи обіцяє запустити одну з двох функцій (переданих як аргументи) в якийсь момент у майбутньому:

- **resolve**: функція зворотного виклику запускається після успішного завершення обробки;
- **reject**: необов'язкова функція зворотного виклику, яка запускається, коли виникає помилка.

Екземпляр класу Promise

Теорія: promise

Promise є екземпляром вбудованого класу **Promise**, і створюється конструктором **new Promise** з однією функцією в якості аргумента: const promise = new Promise(() => {});

Конструктор **new Promise** відразу викликає функцію, що передана як аргумент. Ця функція інформує об'єкт **Promise**, коли **буде завершена** подія, що з ним пов'язана.

При створенні екземпляра об'єкта Promise конструктору передається **функція зворотного виклику**, що називається **виконавцем**. Коли створюється new Promise, вона виконується автоматично. **Виконавець** (зворотний виклик) приймає два аргументи, **resolve и reject, які обидва є колбек-функціями**.

Коли функція-виконавець завершить свою роботу, неважливо – зараз чи пізніше, вона повинна викликати один з цих колбеків:

- **resolve(value)** – якщо код успішно виконався, з результатом value.
- **reject(error)** – якщо виникла помилка, error – об'єкт помилки.

Функція-виконавець з кодом-виробником

Теорія: promise

```
// приклад конструктора проміса і простої функції-виконавця з кодом-“виробником”, що видає результат з затримкою (через setTimeout):

let promise = new Promise(function(resolve, reject) {
 // функція-виробник викликається автоматично, при виклику new Promise

 // через 1 секундну повідомляється що задача виконання з результатом "завершено"
 setTimeout(() => resolve("завершено"), 1000);
});
```

- Функція-виконавець викликається одразу ж при виклику new Promise.
- Функція-виконавець отримує два аргументи: resolve і reject – ці функції вбудовані в JavaScript, тому нам непотрібно їх створювати. Нам слід всього лиш викликати одну з них по готовності.
- Через одну секунду “обробки” виконавець викличе resolve("done"), щоб передати результат. Ця дія змінить стан об’єкта promise (що був створений конструктором new Promise) з "pending" на "fulfilled":

Це приклад успішно виконаної задачі, в результаті ми отримали “виконаний (**fulfilled**)” проміс.

Функція resolve

Практика: promise

Функція **Promise.resolve** повертає promise, що **успішно завершено,** є еквівалентом такого коду:

```
new Promise(function(resolve) {
            resolve();
    }).then(function() {
        return value;
    });
```

Реакція на успішно виконаний проміс:

```
let promise = new Promise(function(resolve, reject) {
 setTimeout(() => resolve("завершено!"), 1000);
});
// метод resolve запустить першу функцію передану в .then
promise.then(
 result => alert(result), // виведе "завершено!" через 1 секунду
 error => alert(error) // не запуститься
);
```

Функція reject

Теорія: promise

Функція **reject(new Error("Oooooй!"))** повертає **неуспішний** promise.
Це корисно, **якщо потрібно обробити помилку**.

Функція **reject(value)** є еквівалентом такого коду:

```
  new Promise(function(resolve, reject) {
     reject(value);
});
```

```
// приклад, коли функція-виконавець повідомить нам, що задача виконана з помилкою:
let promise = new Promise(function(resolve, reject) {
 // через 1 секунду повідомляється, що задача виконана з помилкою
 setTimeout(() => reject(new Error("Oooooй!")), 1000);
});
```

Виклик першого аргумента (reject(...)) переводить стан об’єкта promise в "rejected":

Метод then

Теорія: promise

Обробник, використанний в **.then(handler)** може **створити й повернути проміс.** У цьому випадку інші обробники чекають, поки він виконається, а потім отримають його результат. Наприклад:

```
new Promise(function(resolve, reject) {
 setTimeout(() => resolve(1), 1000);
}).then(function(result) {
 alert(result); // 1
 return new Promise((resolve, reject) => { // (*)
   setTimeout(() => resolve(result * 2), 1000);
 });
}).then(function(result) { // (**)
 alert(result); // 2
 return new Promise((resolve, reject) => {
   setTimeout(() => resolve(result * 2), 1000);
 });
}).then(function(result) {
 alert(result); // 4
});
```

Повернення промісів дозволяє будувати **ланцюжки асинхронних дій**.

Властивості state та result

Теорія: promise

Властивості **state** та **result** – внутрішні властивості об’єкта Promise:

- **state** (стан) - спочатку **"pending"** (очікування), в результаті виконання функції він може змінюватися на: **"fulfilled"** коли викликається метод resolve і на **"rejected"** коли reject.
- **result** (результат) - спочатку **undefined**, далі змінюється на **value**, коли викликається метод resolve(value) або **error** коли reject(error).

Проміс в стані **resolve** чи **reject** називають “**завершеним (settled)**”, на відміну від початкового стану проміса “**в очікуванні (pending)**”.

- Може бути тільки щось одне: або результат або помилка.
- Функція-виконавець може викликати тільки щось одне: resolve або reject.
- Стан проміса може змінитись лише один раз.
- Всі наступні виклики resolve чи reject ігноруються:
- Методи resolve/reject можуть прийняти тільки один аргумент (або жодного).

Методи .then/.catch/.finally

Теорія: promise

Властивості state і result – **внутрішні властивості** об’єкта Promise, тому ми **не маємо до них прямого доступу**. Для обробки результату слід використовувати методи (**функції-споживачі**): **.then/.catch/.finally.**

Об’єкт Promise служить зв’язною ланкою між функцією виконавцем і функціями-споживачами, котрі отримають або результат, або помилку. Функції споживачі можуть **підписатись** на подію отримання результату за допомогою методів **.then та .catch**. Найважливіший і фундаментальний метод – .then:

```
promise.then(
 function(result) { /* обробляє успішне виконання */ },
 function(error) { /* обробляє помилку */ }
);
```

- Перший аргумент метода .then – функція, що викликається, коли проміс успішно виконується, тобто переходить зі стану "pending" в "resolved" і отримує результат.
- Другим аргументом метод .then приймає функцію, що викликається, коли проміс переходить в стан "rejected" і отримує помилку.

Метод catch

Теорія: promise

У випадку якщо щось пішло не так, функції-виконавцю слід викликати метод **reject**. В reject можна передати аргумент будь-якого типу (як і в resolve), але **рекомендується використовувати об'єкт Error** (чи успадкований від нього об'єкт).

```
let promise = new Promise((resolve, reject) => {
 setTimeout(() => reject(new Error("Oooooй!")), 1000);
});
```

Метод .**catch**() визначає функцію, яка викликається, коли трапляється будь-яка помилка. У цей момент більше не виконуватимуться методи .then(). Ви можете мати кілька методів .catch() у всьому ланцюжку для захоплення різних помилок.

```
// .catch(f) те саме що й promise.then(null, f)
promise.catch(alert); // виведе "Error: Oooooй!" через 1 секунду
Виклик .catch(f) – це скорочений варіант .then(null, f).
```

Метод finally

Теорія: promise

По аналогії з блоком finally зі звичайного try {...} catch {...}, у промісів також є свій метод finally. Виклик .finally(f) подібний до .then(f, f), в тому сенсі, що f виконається в будь-якому випадку, коли проміс перейде в стан "виконано (**settled**)" не залежно від того став він resolved чи rejected.

Обробник finally не приймає аргументів. В finally ми не знаємо, як був завершений проміс, успішно чи ні, тому що зазвичай завдання полягає в тому, щоб виконати “загальні” процедури. Обробник finally пропускає результат чи помилку до наступних обробників:

```
new Promise((resolve, reject) => {
 setTimeout(() => resolve("результат"), 2000)
})
 .finally(() => alert("Проміс завершений")) // запускається першим
 .then(result => alert(result)); // <-- .then показує "резульат"
```

Обробник finally також не повинен нічого повертати. Єдиним винятком із цього правила є випадки, коли обробник finally видає помилку і потім ця помилка переходить до наступного обробника замість будь-якого попереднього результату проміса.

Fetch API

Теорія: promise

Fetch API - це API Javascript на основі **promise** для створення асинхронних запитів HTTP. Це чистий, простий, потужний і гнучкий API для отримання/надсилання даних із/на сервер, що є більш потужною та гнучкою заміною XMLHttpRequest.

```
// Простий приклад запиту GET із використанням Fetch API
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(function(response) {
  })
  .catch(function(error) {
  });
```

Fetch API використовує об’єкти **Request** і **Response**, а також концепції CORS і семантику заголовка HTTP Origin. Щоб зробити запит і отримати ресурс, використовують **метод GlobalFetch.fetch()**. Це глобальний метод як у віконному, так і в робочому контекстах, що робить його доступним в будь-якому контексті.

Headers - надає заголовки запиту/відповіді, дозволяє запитувати дані та виконувати різні дії в залежності від результата.

Метод GET

Теорія: promise

За замовчуванням Fetch API використовує метод GET:

```
 fetch(url) // Call the fetch function passing the url of the API as a parameter
.then(function() {
    // Your code for handling the data you get from the API
})
.catch(function() {
    // This is where you run code if the server returns any errors
});
fetch(url) // Виклик функції fetch, передаючи URL-адресу API як параметр
.then(function() {
     // Ваш код для обробки даних, які ви отримуєте з API
})
.catch(функція() {
     // Тут ви запускаєте код, якщо сервер повертає будь-які помилки
});
```

Шлях до ресурсу, який потрібно отримати

Теорія: promise

Метод fetch(), який використовується для отримання ресурсу, приймає один обов'язковий аргумент - шлях до ресурсу, який потрібно отримати. Він повертає Promise, який містить відповідь на цей запит. Також можна додатково передати об'єкт параметрів ініціалізації як другий аргумент.

Базовий синтаксис:

```
let promise = fetch(url, [options])
```

- **url** – URL для відправлення запиту.
- **options** – додаткові параметри: метод, заголовки і т.д.

Без options, це просто GET запит, який завантажує ресурс за адресою url.

Браузер відразу починає робити запит та повертає проміс.

Відповідь на мережевий запит

Теорія: promise

Відповідь містить інформацію про запит та відповідь на мережевий запит, включаючи заголовки, код статусу та повідомлення про статус.
Тіло **Response** має кілька методів:

clone() - метод створює клон відповіді.
redirect() – метод створює нову відповідь, але з іншою URL-адресою.
arrayBuffer() – повертає обіцянку, яка вирішується за допомогою ArrayBuffer.
formData() – повертає обіцянку, але таку, яка вирішується за допомогою об’єкта FormData.
blob() - метод, який вирішує обіцянку за допомогою Blob.
text() – метод, який вирішує обіцянку за допомогою рядка.
json() – метод, який вирішує обіцянку за допомогою JSON.

Налаштування заголовку запиту

Теорія: promise

Для налаштування заголовку запиту потрібно передати об’єкт заголовку разом із запитом.

```
fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  body: JSON.stringify({
    title: 'foo',
    body: 'bar',
    userId: 1
  }),
  headers: { "Content-type": "application/json" }
}).then(function(response){
  return response.json();
}).then(function(data) {
  // data here is JSON object
}).catch(function(error) {
  //
});
```

Процес отримання запиту

Теорія: promise

Процес отримання запиту зазвичай відбувається у два етапи. promise завершується та повертає об’єкт класу Response у якості результату, одразу коли сервер надішле заголовки відповіді. На цьому етапі можна перевірити статус HTTP-запиту, та визначити, чи виконався він успішно, а також переглянути заголовки, проте без тіла запиту.

Проміс закінчується помилкою, якщо fetch не зміг виконати HTTP-запит, наприклад, через помилку мережі або, якщо такого сайту не існує. HTTP-статуси 404 та 500, не викликатимуть помилку. Ми можемо побачити HTTP-статус у властивостях відповіді:

- status – код статуса HTTP-запиту, наприклад, 200.
- ok – логічне значення, котре буде true, якщо код HTTP-статосу в діапазоні 200-299.

```
let response = await fetch(url);
if (response.ok) { // якщо HTTP-статус у діапазоні 200-299
 // отримання тіла запиту
} else {
 alert("HTTP-Error: " + response.status);
}
```

Об’єкт Response

Теорія: promise

```
fetch('https://dummyjson.com/products')
  .then(function(response) {
    console.log(response.headers.get('content-type')); // отримати заголовки відповідей
    console.log(response.headers.get('expires'));
    console.log(response.status);  //  Код статусу відповіді HTTP
    console.log(response.ok); // скорочення для `status` між 200 і 299
    console.log(response.statusText); // повідомлення статусу відповіді, напр. `OK`
    console.log(response.redirected); // перевірити, чи було перенаправлення
    console.log(response.type);      // отримати тип відповіді (`basic`, `cors`)
    console.log(response.url);    // повний шлях до ресурсу
  })
  .catch(function(error) {
    //
  });
```

Отримання тіла запиту

Теорія: promise

Для отримання тіла запиту, потрібно використовувати додатковий виклик методу. Response надає декілька методів, які повертають проміс, для доступу до тіла запиту в різних форматах:

- response.text() – читає відповідь та повертає, як звичайний текст,
- response.json() – декодує відповідь у форматі JSON,
- response.formData() – повертає відповідь, як об’єкт FormData (він буде розглянутий у наступному розділі),
- response.blob() – повертає відповідь, як Blob (бінарні дані з типом),
- response.arrayBuffer() – повертає відповідь, як ArrayBuffer (низькорівневе представлення двійкових даних),

Крім того, response.body це об’єкт ReadableStream, за допомогою якого можна отримувати (зчитувати) тіло відповіді частинами.

Можна вибрати тільки один метод читання відповіді.

Метод json

Теорія: promise

Об’єкт Response має метод json(), що дозволяє отримати дані відповіді як JSON. Метод json() також асинхронний. Отже, це випадок, коли ми повинні викликати дві послідовні асинхронні функції.

```
fetchPromise.then((response) => {
 const jsonPromise = response.json();
 jsonPromise.then((data) => {
   console.log(data[0].title);
 });
});
```

Особливість промісів полягає в тому, що then() сам повертає проміс, який буде завершено результатом переданої йому функції. Це означає, що ми можемо переписати код так:

```
fetchPromise
 .then((response) => response.json())
 .then((data) => {
   console.log(data[0].title);
});
```

Встановлення заголовка запиту в fetch

Теорія: promise

Для встановлення заголовка запиту в fetch, можна використати властивість headers в об’єкті options. Вона містит об’єкт з вихідними заголовками, наприклад:

```
let response = fetch(protectedUrl, {
 headers: {Authentication: 'secret'}
});
```

Заголовки відповіді зберігаются в об’єкті response.headers:

```
let response = await fetch('https://api.github.com/repos/javascript-tutorial/en.javascript.info/commits');
// отримання одного заголовку
alert(response.headers.get('Content-Type')); // application/json; charset=utf-8
// перебір усіх заголовків
for (let [key, value] of response.headers) {
 alert(`${key} = ${value}`);
}
```

Код статусу у відповіді

Теорія: promise

Потрібно перевірити, чи сервер прийняв і зміг обробити запит, перш ніж спробувати його прочитати. Перевіримо код статусу у відповіді та викинемо помилку, якщо статус не «OK»:

```
fetchPromise
.then((response) => {
  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }
  return response.json();
})
.then((data) => {
  console.log(data[0].title);
});
```

Обробка помилок

Теорія: promise

API fetch() може викидати помилку з багатьох причин (наприклад, через відсутність підключення до мережі або URL-адресу було якимось чином неправильно). Для підтримки обробки помилок об’єкти Promise мають метод catch(). Якщо додати catch() у кінець ланцюжка обіцянок, тоді він буде викликаний, коли будь-який із викликів асинхронної функції завершиться помилкою.

```
fetchPromise
.then((response) => {
  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }
  return response.json();
})
.then((data) => {
  console.log(data[0].title);
})
.catch((error) => {
  console.error(`Could not get posts: ${error}`);
});
```

Метод catch

Теорія: promise

Ми можемо використовувати метод catch(), щоб перехопити будь-яку помилку, яка виникає під час виконання запиту. Однак помилка не виникне, якщо запит потрапляє на сервер і повертається, незалежно від того, яку відповідь повернув сервер. Promise, що повертає fetch(), не відхиляє помилки HTTP, навіть якщо код відповіді HTTP 404 або 500.

```
fetch('https://jsonplaceholder.typicode.com/some-invalid-path', {
    credentials: 'include'
  }).then(function(response) {
     if(response.ok) {
      return response.json();
    }
     return Promise.reject(response.status);
  }).catch(function(error) {
    console.error(error)
  });
```

Додавання async

Теорія: promise

Додавання **async** на початку функції робить її асинхронною. Усередині асинхронної функції можна використовувати ключове слово **await** перед викликом функції, яка повертає обіцянку. Це змушує код чекати в цей момент, поки обіцянка не буде вирішена, після чого виконане значення обіцянки розглядається як значення, що повертається, або відхилене значення викидається:

```
async function fetchProducts() {
 try {
   const response = await fetch(
    'https://dummyjson.com/products'
   );
   if (!response.ok) {
     throw new Error(`HTTP error: ${response.status}`);
   }
   const data = await response.json();
   console.log(data[0].name);
 } catch (error) {
   console.error(`Could not get products: ${error}`);
 }}
fetchProducts();
```

Використання async/await

Теорія: promise

Використання **async/await:**

- зовнішній функції має передувати асинхронний оператор **async**
- викликам асинхронних функцій на основі обіцянок має передувати очікування - **await**, щоб забезпечити завершення обробки перед виконанням наступної команди

```
(async () => {
        response = await afetchData(url);
        console.log(response)
})();
```

await ефективно робить кожен виклик так, ніби він синхронний, не затримуючи потік обробки JavaScript. Крім того, асинхронні функції завжди повертають обіцянку, тому їх, у свою чергу, можуть викликати інші асинхронні функції.

Promise

Використання методу fetch.

Динамічний підрахунок товарів

Практика: promise

Динамічний підрахунок:

Показує кількість товарів, які відповідають поточним текстовим та ціновим фільтрам, без врахування вибору самих чекбоксів (щоб користувач бачив, скільки товарів з'явиться, якщо він поставить галочку).

```
<!-- Контентна частина -->
  <main class="content">
    <h2>Товари (<span id="totalProductsCount">0</span>)</h2>
    <div id="productsContainer" class="products-list"></div>
  </main>
```

Пагінація товарів

Практика: promise

Пагінація:

Розбиває відфільтровані результати на сторінки (наприклад, по 3 товари на сторінку) та автоматично скидається на 1-шу сторінку при зміні будь-якого фільтра.

```
<!-- Контентна частина -->
  <main class="content">
    <h2>Товари (<span id="totalProductsCount">0</span>)</h2>
    <div id="productsContainer" class="products-list"></div>
    <!-- Контейнер для кнопок сторінок -->
    <div id="paginationContainer" class="pagination"></div>
  </main>
```

Кнопка очищення

Практика: promise

Кнопка очищення: Скидає всі інпути, чекбокси та селектори до початкового стану.

```
<!-- Сортування -->
      <div class="filter-group">
        <h3>Сортування</h3>
        <select id="sortSelect" class="sort-select">
          <option value="default">За замовчуванням</option>
          <option value="price-asc">Від дешевих до дорогих</option>
          <option value="price-desc">Від дорогих до дешевих</option>
          <option value="name-asc">За назвою (А-Я)</option>
        </select>
      </div>
      <button id="clearFiltersBtn" class="clear-btn">Очистити всі фільтри</button>
  </aside>
```

Стилі

Практика: promise

```
body {
    font-family: sans-serif;
    display: flex;
    gap: 20px;
    padding: 20px;
    margin: 0;
    background: #f9f9f9;
}
.sidebar {
    width: 260px;
    padding: 20px;
    border: 1px solid #ddd;
    border-radius: 8px;
    background: #fff;
    display: flex;
    flex-direction: column;
    gap: 20px;
}
```

Стилі

Практика: promise

```
.sidebar h3 {
    margin: 0 0 10px 0;
    font-size: 1.1em;
    border-bottom: 1px solid #eee;
    padding-bottom: 5px;
}

.filter-group {
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.price-inputs {
    display: flex;
    gap: 10px;
}
```

Стилі

Практика: promise

```
.price-inputs input {
    width: 100%;
    padding: 5px;
}
.search-input,
.sort-select {
    padding: 8px;
    width: 100%;
    box-sizing: border-box;
}
.count-badge {
    background: #eee;
    font-size: 0.8em;
    padding: 2px 6px;
    border-radius: 10px;
    float: right;
    color: #555;
}
```

Стилі

Практика: promise

```
.content {
    flex-grow: 1;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    min-height: 85vh;
}
.products-list {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
    gap: 15px;
    margin-bottom: 20px;
}
.product-card {
    padding: 15px;
    border: 1px solid #ddd;
    border-radius: 8px;
    background: #fff;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.02);
}
```

Стилі

Практика: promise

```
.product-price {
    font-weight: bold;
    color: #2c3e50;
    margin: 8px 0;
}
.product-category {
    color: #888;
    font-size: 0.85em;
}

/* СТИЛІ ПАГІНАЦІЇ */
.pagination {
    display: flex;
    gap: 5px;
    justify-content: center;
    margin-top: auto;
    padding: 20px 0;
}
```

Стилі

Практика: promise

```
.page-btn {
    padding: 8px 12px;
    border: 1px solid #ddd;
    background: #fff;
    cursor: pointer;
    border-radius: 4px;
}

.page-btn.active {
    background: #007bff;
    color: white;
    border-color: #007bff;
}
.page-btn:disabled {
    background: #eee;
    cursor: not-allowed;
    color: #aaa;
}
```

Стилі

Практика: promise

```css
/* КНОПКА СКИДАННЯ */
.clear-btn {
    padding: 10px;
    background: #dc3545;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
}

.clear-btn:hover {
    background: #bd2130;
}
```

Глобальний стан пагінації

Практика: promise

```
// 1. Глобальні змінні стану додатка

let products = []; // Порожній масив, сюди запишуться дані після fetch

let currentPage = 1;

const itemsPerPage = 3; // Скільки товарів виводити на одній сторінці
```

DOM Елементи

Практика: promise

```
// 2. DOM Елементи
const container = document.getElementById('productsContainer');

const searchInput = document.getElementById('searchInput');
const checkboxes = document.querySelectorAll('.category-checkbox');
const priceMinInput = document.getElementById('priceMin');
const priceMaxInput = document.getElementById('priceMax');
const sortSelect = document.getElementById('sortSelect');

const clearFiltersBtn = document.getElementById('clearFiltersBtn');
const paginationContainer = document.getElementById('paginationContainer');
const totalProductsCountHTML = document.getElementById('totalProductsCount');
```

Функція динамічного підрахунку кількості товарів

Практика: promise

```
// Динамічний підрахунок кількості товарів (Бейджі)
function updateBadges(searchQuery, minPrice, maxPrice) {
    const baseFiltered = products.filter(product => {
        const matchesSearch = product.name.toLowerCase().includes(searchQuery);
        const matchesPrice = product.price >= minPrice && product.price <= maxPrice;
        return matchesSearch && matchesPrice;
    });

    // Оновлюємо бейджі відповідно до нових категорій з сервера
    const categories = ['beauty', 'fragrances', 'groceries'];
    categories.forEach(cat => {
        const badge = document.getElementById(`count-${cat}`);
        if (badge) {
            const count = baseFiltered.filter(p => p.category === cat).length;
            badge.textContent = count;
        }
    });
}
```

Рендер кнопок пагінації

Практика: promise

```
//  Рендер кнопок пагінації

function renderPagination(totalItems) {
   paginationContainer.innerHTML = '';
   const totalPages = Math.ceil(totalItems / itemsPerPage);

   if (totalPages <= 1) return;

   const prevBtn = document.createElement('button');
   prevBtn.className = 'page-btn';
   prevBtn.textContent = '«';
   prevBtn.disabled = currentPage === 1;
   prevBtn.addEventListener('click', () => { currentPage--; updateInterface(false); });
   paginationContainer.appendChild(prevBtn);
```

Рендер кнопок пагінації

Практика: promise

```
  for (let i = 1; i <= totalPages; i++) {
      const pageBtn = document.createElement('button');
      pageBtn.className = `page-btn ${i === currentPage ? 'active' : ''}`;
      pageBtn.textContent = i;
      pageBtn.addEventListener('click', () => { currentPage = i; updateInterface(false); });
      paginationContainer.appendChild(pageBtn);
  }

  const nextBtn = document.createElement('button');
  nextBtn.className = 'page-btn';
  nextBtn.textContent = '»';
  nextBtn.disabled = currentPage === totalPages;
  nextBtn.addEventListener('click', () => { currentPage++; updateInterface(false); });
  paginationContainer.appendChild(nextBtn);
}
```

Рендер карток товарів

Практика: promise

```
// Рендер карток товарів
function renderProducts(items) {
  container.innerHTML = '';
  if (items.length === 0) {
    container.innerHTML = '<p style="grid-column: 1/-1;">Товарів не знайдено.</p>';
    return;
  }
  items.forEach(product => {
    const card = document.createElement('div');
    card.className = 'product-card';
    card.innerHTML = `
    <h4>${product.name}</h4>
    <p class="product-price">${product.price} грн</p>
    <p class="product-category">Категорія: ${product.category}</p>
  `;
    container.appendChild(card);
  });
}
```

Функція автоматичного очищення фільтрів

Практика: promise

```
// Функція автоматичного очищення фільтрів
function clearFilters() {
    searchInput.value = '';
    priceMinInput.value = '';
    priceMaxInput.value = '';
    sortSelect.value = 'default';
    checkboxes.forEach(cb => cb.checked = false);

    window.history.replaceState(null, '', window.location.pathname);
    updateInterface(true);
}
```

Головна функція оновлення інтерфейсу

Практика: promise

```
// Головна функція оновлення інтерфейсу

function updateInterface(resetPage = true) {
  // Якщо дані ще не завантажилися з сервера, зупиняємо виконання
  if (products.length === 0) return;

  if (resetPage) currentPage = 1;

  const searchQuery = searchInput.value.toLowerCase().trim();
  const minPrice = parseFloat(priceMinInput.value) || 0;
  const maxPrice = parseFloat(priceMaxInput.value) || Infinity;
  const sortType = sortSelect.value;
  const activeCategories = Array.from(checkboxes).filter(cb => cb.checked).map(cb => cb.value);

  syncFiltersToURL(searchInput.value.trim(), minPrice, maxPrice, sortType, activeCategories);
  updateBadges(searchQuery, minPrice, maxPrice);
```

Головна функція оновлення інтерфейсу

Практика: promise

```
  let filtered = products.filter(product => {
      const matchesSearch = product.name.toLowerCase().includes(searchQuery);
      const matchesCategory = activeCategories.length === 0 ||
activeCategories.includes(product.category);
      const matchesPrice = product.price >= minPrice && product.price <= maxPrice;
      return matchesSearch && matchesCategory && matchesPrice;
  });
  totalProductsCountHTML.textContent = filtered.length;
  if (sortType === 'price-asc') filtered.sort((a, b) => a.price - b.price);
  else if (sortType === 'price-desc') filtered.sort((a, b) => b.price - a.price);
  else if (sortType === 'name-asc') filtered.sort((a, b) => a.name.localeCompare(b.name, 'uk'));
  const startIndex = (currentPage - 1) * itemsPerPage;
  const endIndex = startIndex + itemsPerPage;
  const paginatedItems = filtered.slice(startIndex, endIndex);
  renderProducts(paginatedItems);
  renderPagination(filtered.length);
}
```

Слухачі подій

Практика: promise

```
// Слухачі подій
searchInput.addEventListener('input', () => updateInterface(true));
priceMinInput.addEventListener('input', () => updateInterface(true));
priceMaxInput.addEventListener('input', () => updateInterface(true));
sortSelect.addEventListener('change', () => updateInterface(true));
checkboxes.forEach(cb => cb.addEventListener('change', () => updateInterface(true)));
clearFiltersBtn.addEventListener('click', clearFilters);
```

Категорії в HTML (для тестування з реальним API)

Практика: promise

```
<!-- Фільтр за категоріями →\

      <div class="filter-group">
        <h3>Категорії</h3>
        <label><input type="checkbox" value="beauty" class="category-checkbox"> Краса (beauty) <span
              class="count-badge" id="count-beauty">0</span></label>
        <label><input type="checkbox" value="fragrances" class="category-checkbox"> Парфуми
(fragrances) <span
              class="count-badge" id="count-fragrances">0</span></label>
        <label><input type="checkbox" value="groceries" class="category-checkbox"> Продукти
(groceries) <span
              class="count-badge" id="count-groceries">0</span></label>
      </div>
```

АСИНХРОННА ФУНКЦІЯ ЗАВАНТАЖЕННЯ ДАНИХ

Практика: promise

```
// 3. АСИНХРОННА ФУНКЦІЯ ЗАВАНТАЖЕННЯ ДАНИХ (FETCH)

async function fetchProducts() {
   try {
      // Показуємо користувачу індикатор завантаження
      container.innerHTML = '<p style="grid-column: 1/-1; text-align: center;">Завантаження товарів...</p>';

      // Запитуємо дані з сервера (ліміт 30 товарів для тесту)
      const response = await fetch('https://dummyjson.com/products');

      if (!response.ok) {
         throw new Error(`Помилка HTTP: ${response.status}`);
      }

      const data = await response.json();
```

АСИНХРОННА ФУНКЦІЯ ЗАВАНТАЖЕННЯ ДАНИХ

Практика: promise

```
/       // МАТРИЦЯ ТРАНСФОРМАЦІЇ: Приводимо серверні дані до нашого формату
        // DummyJSON має поля: title, category, price, id
        products = data.products.map(item => ({
            id: item.id,
            name: item.title,
            category: item.category, // beauty, fragrances, groceries тощо
            price: Math.round(item.price * 41) // Конвертуємо умовні долари в грн для реалістичності
        }));
        // Після успішного завантаження запускаємо логіку інтерфейсу
        loadFiltersFromURL();
        updateInterface(false);
    } catch (error) {
        console.error("Не вдалося завантажити дані:", error);

        container.innerHTML = `<p style="color: red; grid-column: 1/-1;">Сталася помилка при
завантаженні товарів: ${error.message}</p>`;
    }
}
```

Функція завантаження стану фільтрів з URL

Практика: promise

```
// Функція завантаження стану фільтрів з URL
function loadFiltersFromURL() {
    const params = new URLSearchParams(window.location.search);
    if (params.has('search')) searchInput.value = params.get('search');
    if (params.has('minPrice')) priceMinInput.value = params.get('minPrice');
    if (params.has('maxPrice')) priceMaxInput.value = params.get('maxPrice');
    if (params.has('sort')) sortSelect.value = params.get('sort');
    if (params.has('page')) currentPage = parseInt(params.get('page')) || 1;

    if (params.has('categories')) {
        const activeCats = params.get('categories').split(',');
        checkboxes.forEach(cb => {
            if (activeCats.includes(cb.value)) cb.checked = true;
        });
    }
}
```

Функція синхронізації стану з URL

Практика: promise

```
// Функція синхронізації стану з URL
function syncFiltersToURL(searchQuery, minPrice, maxPrice, sortType, activeCategories) {
   const params = new URLSearchParams();
   if (searchQuery) params.set('search', searchQuery);
   if (minPrice > 0) params.set('minPrice', minPrice);
   if (maxPrice && maxPrice !== Infinity) params.set('maxPrice', maxPrice);
   if (sortType !== 'default') params.set('sort', sortType);
   if (currentPage > 1) params.set('page', currentPage);
   if (activeCategories.length > 0) params.set('categories', activeCategories.join(','));

   const newRelativePathQuery = window.location.pathname + (params.toString() ? '?' + params.toString()
: '');
   window.history.replaceState(null, '', newRelativePathQuery);
}
```

СТАРТ ПРОГРАМИ

Практика: promise

```
// --- СТАРТ ПРОГРАМИ ---
// Замість миттєвого виклику updateInterface, спочатку ініціюємо fetch-запит до API
fetchProducts();
```

1. async/await та Блок try...catch: код став асинхронним. try виконує запит, а catch перехоплює будь-які помилки (відсутність інтернету, падіння сервера, неправильний URL) та безпечно виводить повідомлення про помилку на екран користувача без "падіння" всього сайту.
2. Захист if (products.length === 0) return;: Оскільки fetch займає певний час, користувач може спробувати клацати фільтри ще до того, як прийшла відповідь від сервера. Цей рядок захищає функцію updateInterface від помилок роботи з порожнім масивом.
3. Етап мапінгу (.map()): Ми не працюємо з "сирими" даними з сервера. Ми відразу перетворюємо їх на зручну нам структуру, що дозволяє легко замінити це API на будь-яке інше у майбутньому - достатньо буде лише змінити правила у методі map.

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua